Metody Optymalziacji - laboratorium identyfikacja odwróconego wahadła

1 Funkcja `fCreateInvertedPendulumStateSpaceModel`

```
function model = fCreateInvertedPendulumStateSpaceModel(m, l, I)
M = 0.5;
b = 0.1;
g = 9.8;
```

Opis: Definiujemy znane parametry modelu: masa wózka M, współczynnik tarcia b i przyspieszenie ziemskie g. Parametry m, l, I są wejściem funkcji – to właśnie one będą estymowane w zadaniu optymalizacji.

```
den = I*(M+m)+M*m*l^2;
```

Opis: Wyznaczamy mianownik wspólny dla macierzy stanu, który pojawia się przy przekształceniu równań ruchu.Jest to wyrażenie wynikające z eliminacji przyspieszeń z równań dynamiki układu.

```
A = [0      1               0            0;
     0 -(I+m*l^2)*b/den  (m^2*g*l^2)/den   0;
     0      0               0            1;
     0 -(m*l*b)/den       m*g*l*(M+m)/den  0];
```

Opis: Macierz stanu A opisuje dynamikę układu w przestrzeni stanów. Każdy element macierzy pochodzi z linearyzowanych równań ruchu wahadła i wózka.

```
B = [     0;
     (I+m*l^2)/den;
          0;
        m*l/den];
```

Opis: Macierz B opisuje wpływ wejścia u (siły przyłożonej do wózka) na stany układu.

```
C = [1 0 0 0;
     0 0 1 0];
D = [0;
     0];
```

Opis: Macierze wyjścia C i D określają jakie sygnały stanu są obserwowane: x – położenie wózka i φ – kąt wahadła. Nie ma bezpośredniego sprzężenia wejścia do wyjścia, więc $D = 0$.

```
states = {'x' 'x_dot' 'phi' 'phi_dot'};
inputs = {'u'};
outputs = {'x'; 'phi'};

model = ss(A,B,C,D,'statename',states,'inputname',inputs,'
    outputname',outputs);
end
```

Opis: Definiujemy nazwy stanów, wejść i wyjść dla czytelności. Funkcja zwraca obiekt 'ss' MATLAB, który reprezentuje model w przestrzeni stanów.

2 Funkcja `fSimulateModel`

```
function [x, phi] = fSimulateModel(m, l, I)
    model = fCreateInvertedPendulumStateSpaceModel(m, l, I);
```

Opis: Tworzymy model stanu z zadanymi parametrami m, l, I.

```
load dane1_wah_grupa1.mat
u = dane1.um(:,1);
t = dane1.t(:,1);
```

Opis: Wczytujemy dane pomiarowe: sygnał sterujący u oraz wektor czasu t.

```
out = lsim(model, u, t);
x = out(:,1);
phi = out(:,2);
end
```

Opis: 'lsim' symuluje odpowiedź modelu na dane wejściowe $u(t)$. Zwracamy dwie kolumny odpowiedzi: $x(t)$ – pozycję wózka i $\varphi(t)$ – kąt wahadła.

3 Funkcja `fObjectiveFunctionReturnScalar`

```
function error = fObjectiveFunctionReturnScalar(lambda, xRef,
    phiRef)
m = lambda(1);
l = lambda(2);
I = lambda(3);

[x, phi] = fSimulateModel(m, l, I);
```

Opis: Funkcja pobiera wektor parametrów λ i sygnały referencyjne x_{ref}, ϕ_{ref}. Symuluje model dla tych parametrów.

```
error = sum([ (xRef - x).^2;
              (phiRef - phi).^2 ]);
end
```

Opis: Obliczamy sumę kwadratów uchybu dla obu sygnałów. Zwracany jest **skalar**, który może być użyty w funkcjach 'fminsearch' lub 'fmincon'.

4 Funkcja `fObjectiveFunctionReturnVector`

```
function errorVector = fObjectiveFunctionReturnVector(lambda, xRef,
    phiRef)
m = lambda(1);
l = lambda(2);
I = lambda(3);

[x, phi] = fSimulateModel(m, l, I);

errorVector = [ xRef - x;
                phiRef - phi ];
end
```

Opis: Zwraca wektor reszt, a nie skalar. Wektor reszt jest wymagany do użycia funkcji ‘lsqnonlin‘, która minimalizuje normę 2 tego wektora.

5 Funkcja `fPlotResponseComparison`

```
function fPlotResponseComparison(m, l, I)
load dane1_wah_grupa1.mat
xRef = dane1.ym(:,1);
phiRef = dane1.ym(:,2);
time = dane1.t(:,1);

[x, phi] = fSimulateModel(m, l, I);
```

Opis: Wczytujemy sygnały referencyjne i czas, następnie symulujemy model z zadanymi parametrami.

```
subplot(2,1,1);
xlabel('Time [s]');
ylabel('Cart position');
p1 = plot(time, xRef, time, x, '--');
p1(1).LineWidth = 2;
p1(2).Marker = 'o';
legend('Reference cart position', 'Estimated cart position');
```

Opis: Rysujemy pozycję wózka: sygnał referencyjny vs symulacja. Formatowanie linii i markerów poprawia czytelność wykresu.

```
subplot(2,1,2);
xlabel('Time [s]');
ylabel('Angle from vertical');
p2 = plot(time, phiRef, time, phi, '--');
p2(1).LineWidth = 2;
p2(2).Marker = 'o';
legend('Reference angle', 'Estimated angle');
end
```

Opis: Analogicznie rysujemy kąt wahadła. Funkcja pozwala wizualnie ocenić jakość estymacji parametrów.

6 Skrypt główny - funkcje pomocnicze

```
clear all;
clc;
addpath(genpath('System_identification'));
```

Opis: clear all usuwa wszystkie zmienne z pamięci MATLAB. clc czyści okno poleceń. addpath(genpath(...)) dodaje folder System identification i wszystkie podfoldery do ścieżki MATLAB, aby funkcje pomocnicze były dostępne.

```
load dane1_wah_grupa1.mat

xRef = dane1.ym(:,1);
phiRef = dane1.ym(:,2);
```

Opis: Wczytujemy dane pomiarowe z pliku .mat. Tworzymy wektory referencyjne: xRef – położenie wózka, phiRef – kąt wahadła.

```
objective = @(lambda)(fObjectiveFunctionReturnVector(lambda, xRef,
    phiRef));
```

Opis: Tworzymy funkcję anonimową MATLAB 'objective'. Dla każdego wektora parametrów lambda = [m, l, I] funkcja zwraca wektor reszt między sygnałami referencyjnymi a symulowanymi.

```
lambda_opt = lsqnonlin(objective, [0.4, 1, 0.35])
```

Opis: Wywołujemy funkcję lsqnonlin, aby dopasować parametry lambda. [0.4, 1, 0.35] – wektor początkowy estymowanych parametrów. lsqnonlin minimalizuje normę 2 wektora reszt zwracanego przez funkcję objective. Zwraca optymalne wartości parametrów.

```
[x, phi] = fSimulateModel(lambda_opt(1), lambda_opt(2), lambda_opt
    (3))
```

Opis: Symulujemy model odwróconego wahadła dla estymowanych parametrów. Otrzymujemy wektory x i phi – przewidywane odpowiedzi układu.

```
fPlotResponseComparison(lambda_opt(1), lambda_opt(2),lambda_opt(3))
```

Opis: Wywołujemy funkcję rysującą porównanie odpowiedzi układu: sygnał referencyjny z pomiarów, odpowiedź modelu dla parametrów estymowanych. Ułatwia to wizualną ocenę jakości dopasowania parametrów.